L9A: Environment Model

CS1101S: Programming Methodology

Martin Henz

14th October, 2026

Version 1.0 — Built: 2026-10-13 23:05:55 SGT

State (3.1)

Environment Model (3.2 and 3.3)

State (3.1)

Environment Model (3.2 and 3.3)

Recap: Assignment

- The **declaration assignment** gives `x` its first value.
- The **reassignment** evaluates the right-hand side, then changes the binding of `x`.
- `x` now evaluates to 2.

```
x = 1
x = x + 1
print(x)
```



Recap: Reassignment: Pros

- Reassignment allows us to create objects with state.
- State allows objects to behave differently over time.

Recap: Reassignment: Cons

- Harder to reason about programs
  - Harder to debug
  - Harder to verify correctness
- The substitution model of evaluation breaks down!
  - It is not powerful enough to explain state.
  - We need a more sophisticated model: the environment model.

Recap: Why Substitution Model Breaks Down?

- The substitution model treats a name as referring to a fixed value.
  - Its value will not change.
  - Therefore, the value can be substituted for the name.
- Reassignment instead treats a variable as a "container" holding a value.
  - The contents of the container may change over time.
  - The container is maintained in a structure called an environment.

Environments & Environment Model

- Environments
  - Structures maintained by the interpreter to track the values of names during program evaluation
  - Locations for storing values associated with names
- Environment model
  - A mental model for environments
  - A set of rules for managing environments
  - A graphical representation: environment diagrams

Environment Model (3.2 and 3.3)

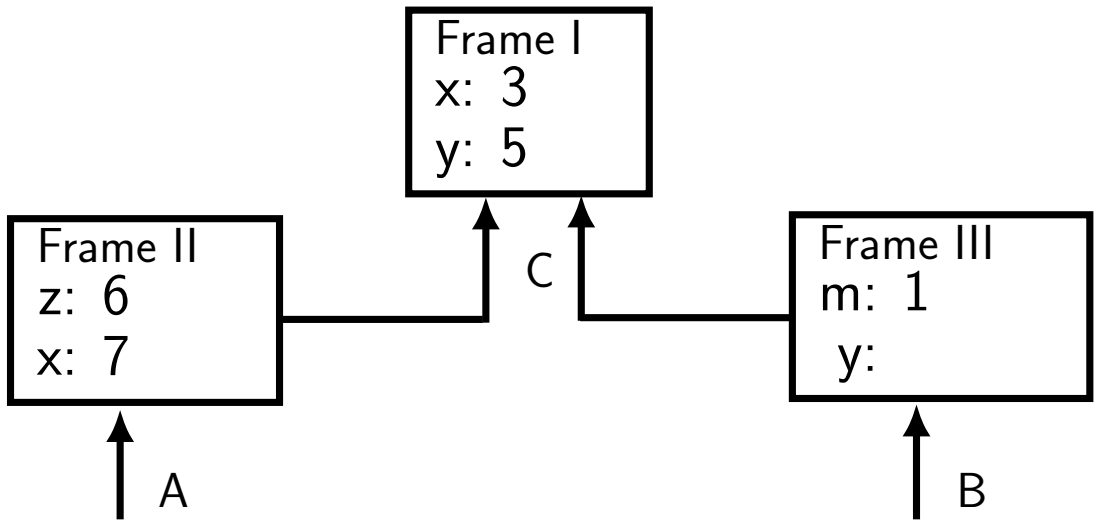
Environments: Concept

- An environment determines the context in which an expression is evaluated.
- Every expression is evaluated with respect to the **current** environment.

Environments: Details

- An environment is a sequence of frames.
- Each frame contains bindings of values to names.
- A frame points to its enclosing environment, the next one in the sequence.
  - Exception: The built-ins frame has no enclosing environment.
- Extending an environment means adding a new frame "inside" it.

Example Environments



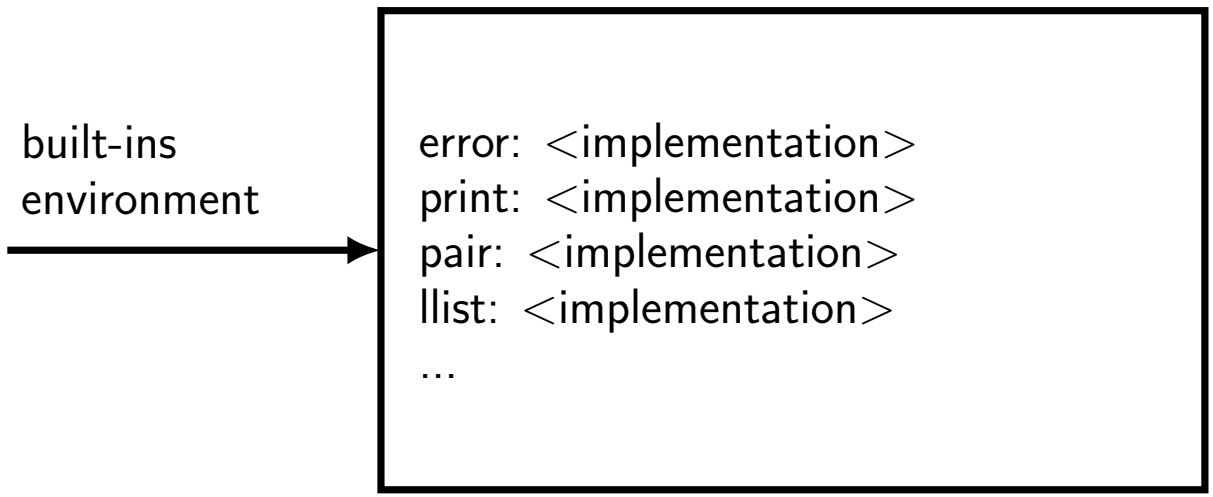
- A, B, and C are environments.
- The value of `x` in A is 7.
- The value of `x` in B is 3.
- The binding in Frame II shadows the binding in Frame I.
- The variable `y` is bound but unassigned in B.

Accessing an Environment

- To evaluate a name, look up its value in the first frame of the current environment.
- If it is not there, continue through the enclosing environments.

Built-ins Environment

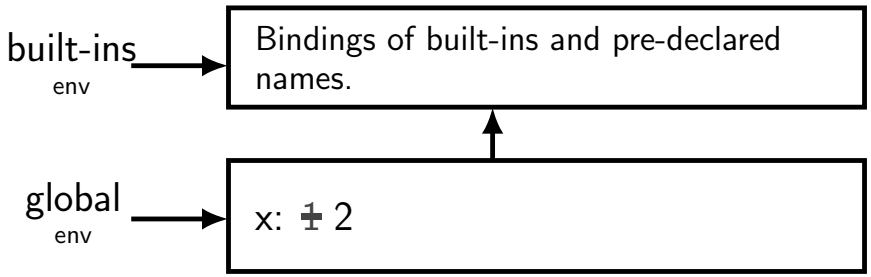
- The built-ins environment is one frame containing built-ins and pre-declared names.
- Pre-declared function names are bound to primitive implementations.



Example: Top-Level Assignments

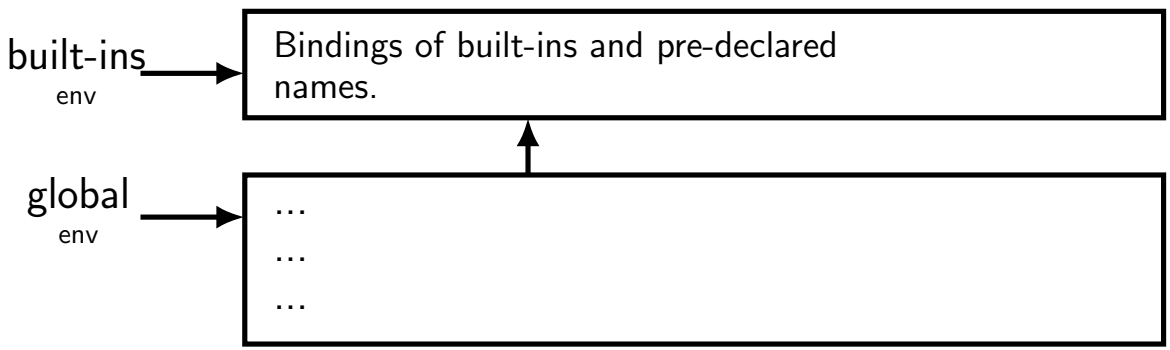
```
x = 1
x = x + 1
print(x)
```

output: 2



The Global Environment

- Evaluating the user program creates the global environment.
  - Its frame will contain the user program's top-level bindings.
  - It directly extends the built-ins environment.
  - Global declaration assignments add bindings to the global frame.

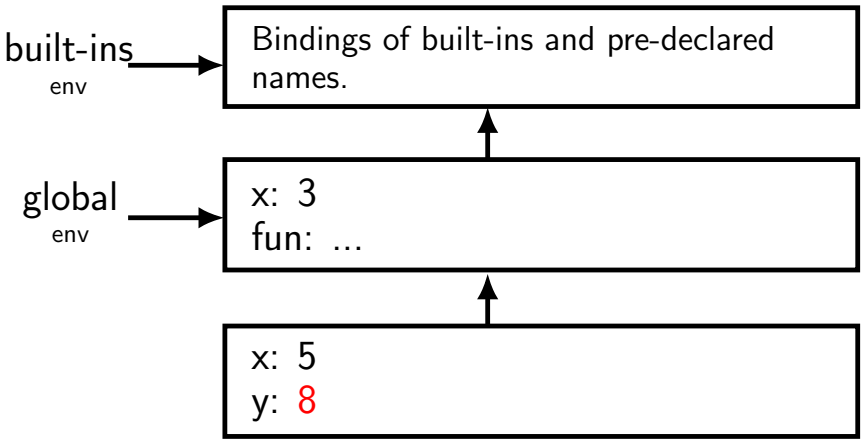


Example: A Function Call Creates a Frame

```
x = 3

def fun():
    x = 5
    y = 8

fun()
```



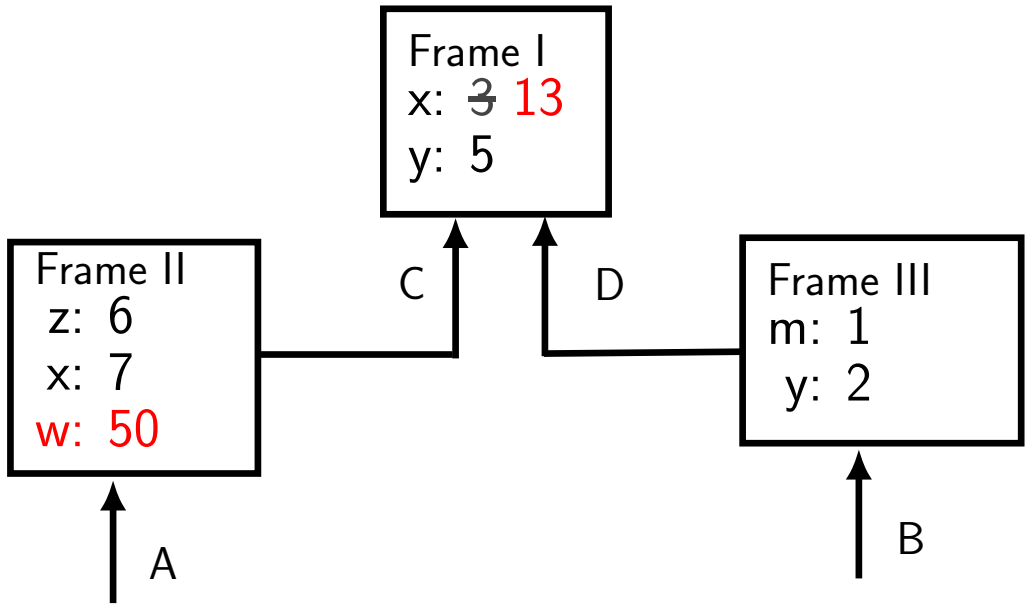
Frame of a Function Call

- Start with the environment stored in the function object.
- Extend it with a new frame.
  - Bind each parameter to its argument value.
  - Bind each local variable, initially unassigned.
- Local variables are the names that occur in any assignment in the body, except if the name is declared global or nonlocal in that body.
- Evaluate the function body in that environment.
- For clarity, empty frames may be omitted from our diagrams.

Evaluation of Assignments

- When a function body is entered, its local names are placed in the call frame, initially unassigned.
- To evaluate `name = expression`:
  - Evaluate `expression`.
  - Assign the resulting value to the selected binding.
- Reading a name before it has been assigned signals an error.

Example Environments



- Evaluate in B: `x = x + 10`.
- The value of `x` in Frame I changes from 3 to 13.

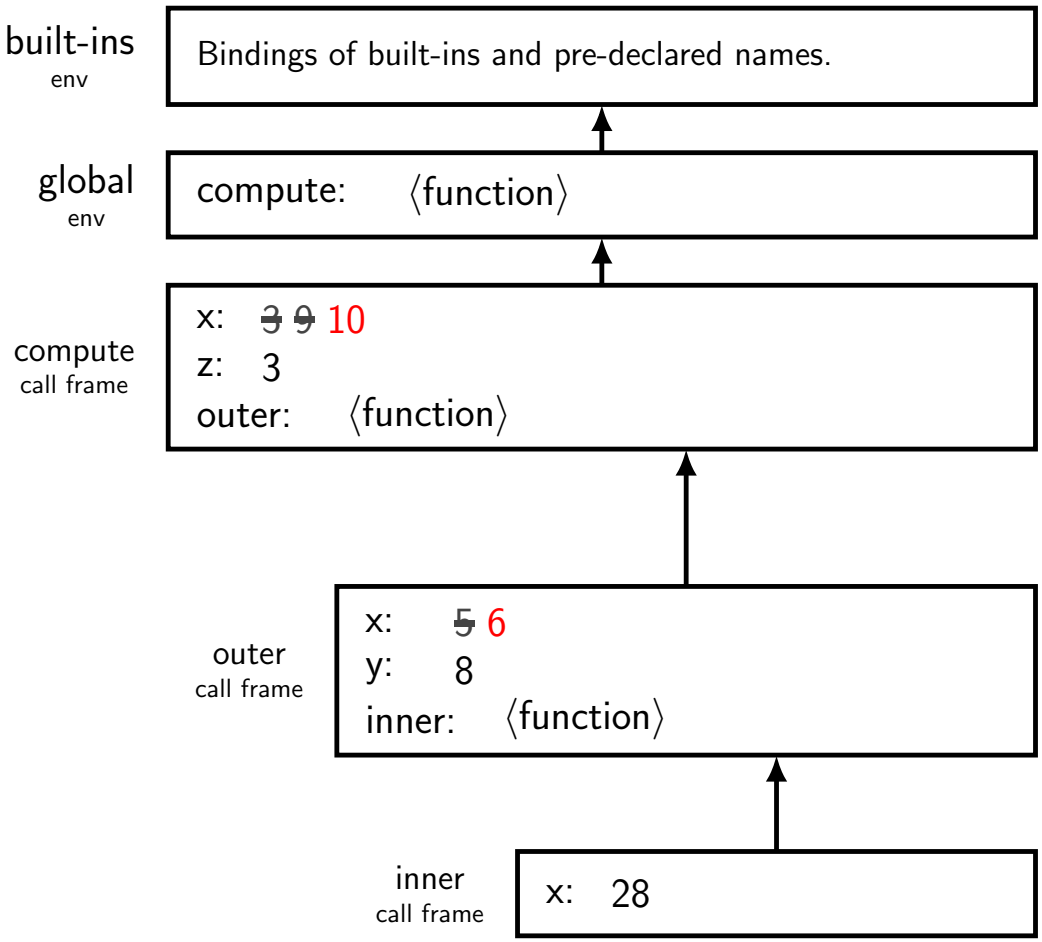
Function-Call Trace

- The trace focuses on these nested function calls:



```
def outer():
    x = 5
    y = 8
    def inner():
        x = 20 + y
        return x
    inner()
    x = x + 1
    return x
return outer()
```

Environment Model Trace




```
def compute(x):
    z = x
    x = z * z
    x = x + 1
    def outer():
        x = 5
        y = 8
        def inner():
            x = 20 + y
            return x
        inner()
        x = x + 1
        return x
    return outer()

compute(3)
```

Evaluation of Lambda Expressions

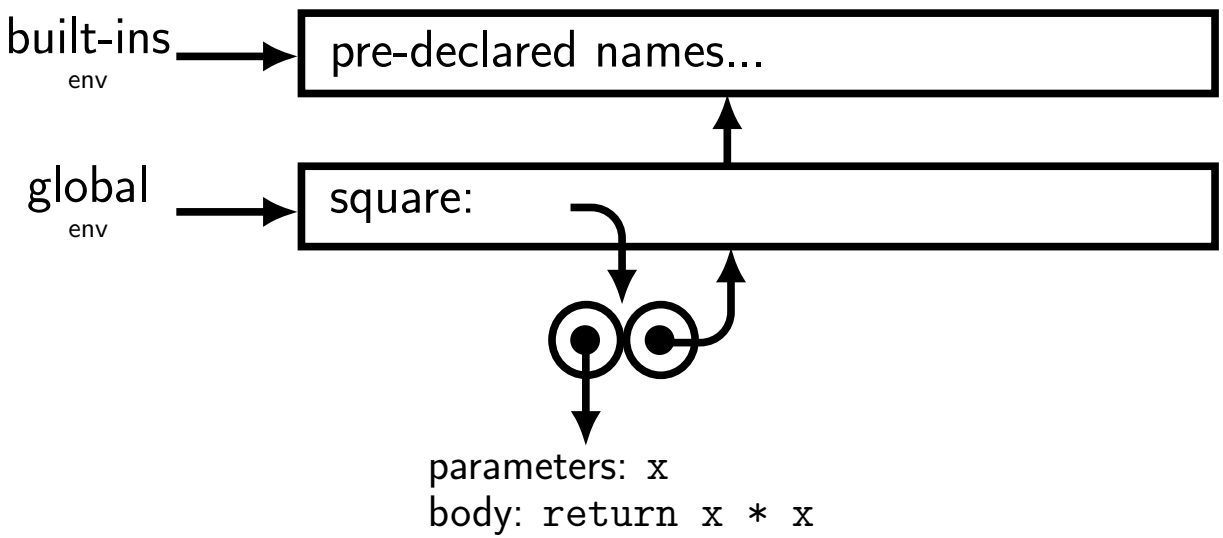
- A lambda expression such as `lambda var1, var2, ...: expression` creates a function object represented by two circles.
  - One circle points to the function's parameters and body.
  - The other points to the environment in which the lambda was evaluated.
  - The function therefore remembers the environment in which it was created.

Example for Evaluating Lambda Expressions

```
square = lambda x: x * x
print(square(5))
```



- After evaluating the assignment in the program environment:



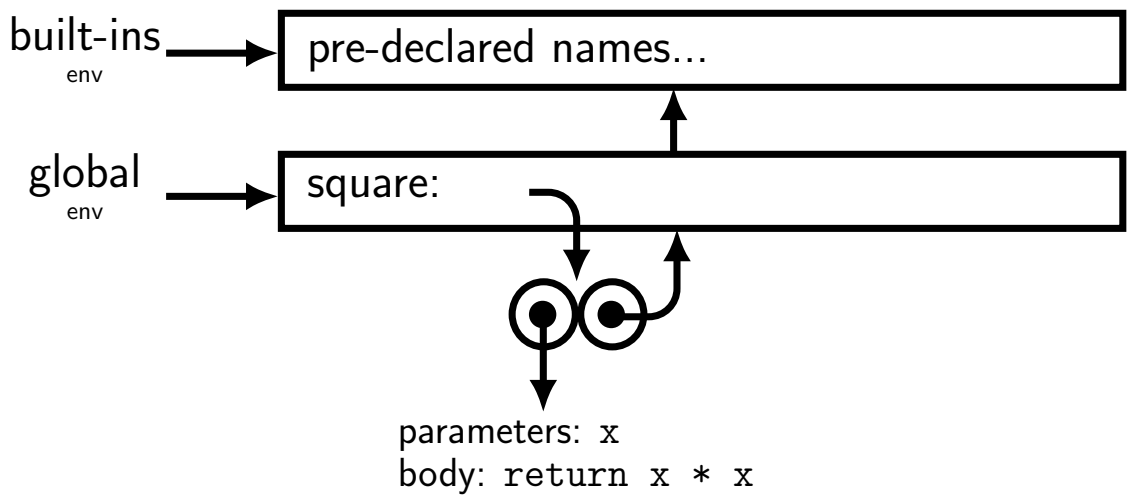
Example for Evaluating Function Definitions

```
def square(x):
    return x * x
```

which is equivalent to

```
square = lambda x: x * x
```

- After evaluating the function definition in the program environment:



Evaluation of Function Applications

- To evaluate a function application:

```
fun(arg1, arg2, ..., argn)
```

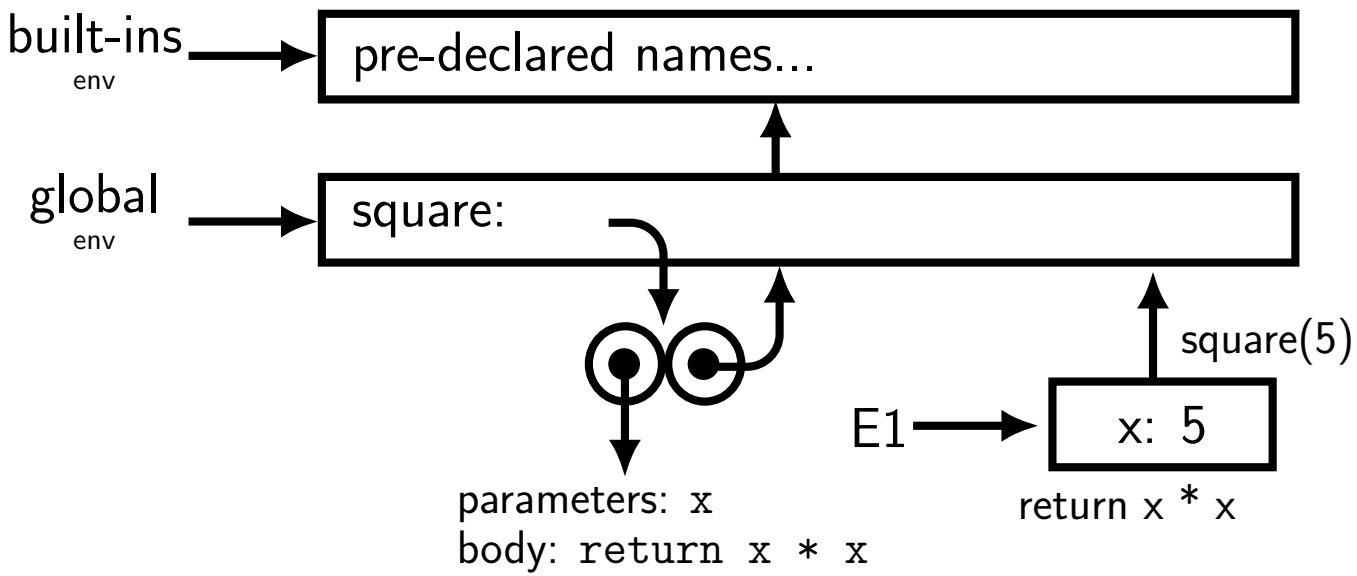
  - Evaluate the subexpressions in the current environment.
  - Apply the value of `fun` to the values of the argument subexpressions.

Applying a Function Value to Argument Values

- Take the environment stored in the function object.
- Extend it with a call frame.
- Bind the parameters to the argument values.
- Bind each local variable, initially unassigned.
- Evaluate the function body in the resulting environment.

Example: square(5)

- Evaluate `square` and 5 in the program environment.
- Create a new frame that points to the environment of `square`.
- Bind the value 5 to parameter `x` in this frame.
- Evaluate the body `return x * x`.



Example with a Local Variable

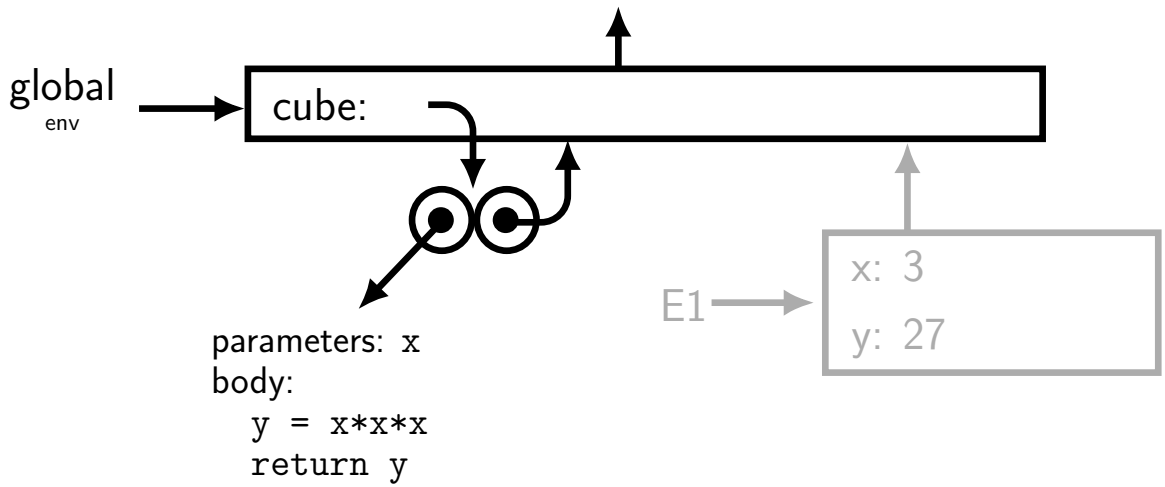
- The function `cube` has a local variable `y`.

```
def cube(x):
    y = x * x * x
    return y
print(cube(3))
```



cube Example

- Environment after evaluating `cube(3)`.




```
def cube(x):
    y = x * x * x
    return y
print(cube(3))
```

Example with Recursion

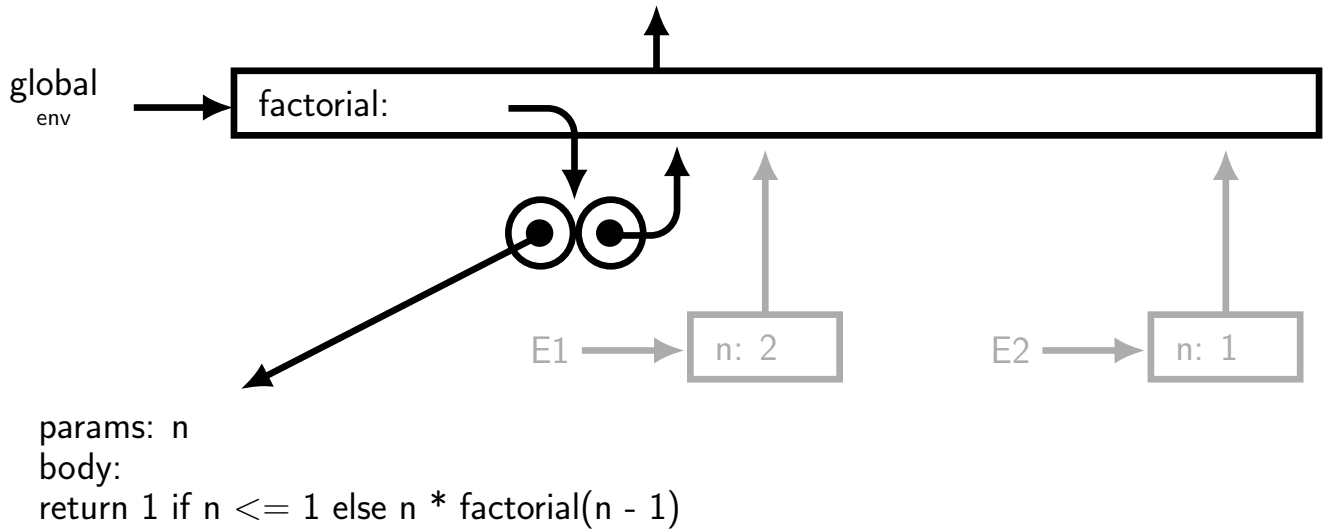
- A recursive definition of `factorial`:

```
def factorial(n):
    if n <= 1:
        return 1
    else:
        return n * factorial(n - 1)
print(factorial(2))
```



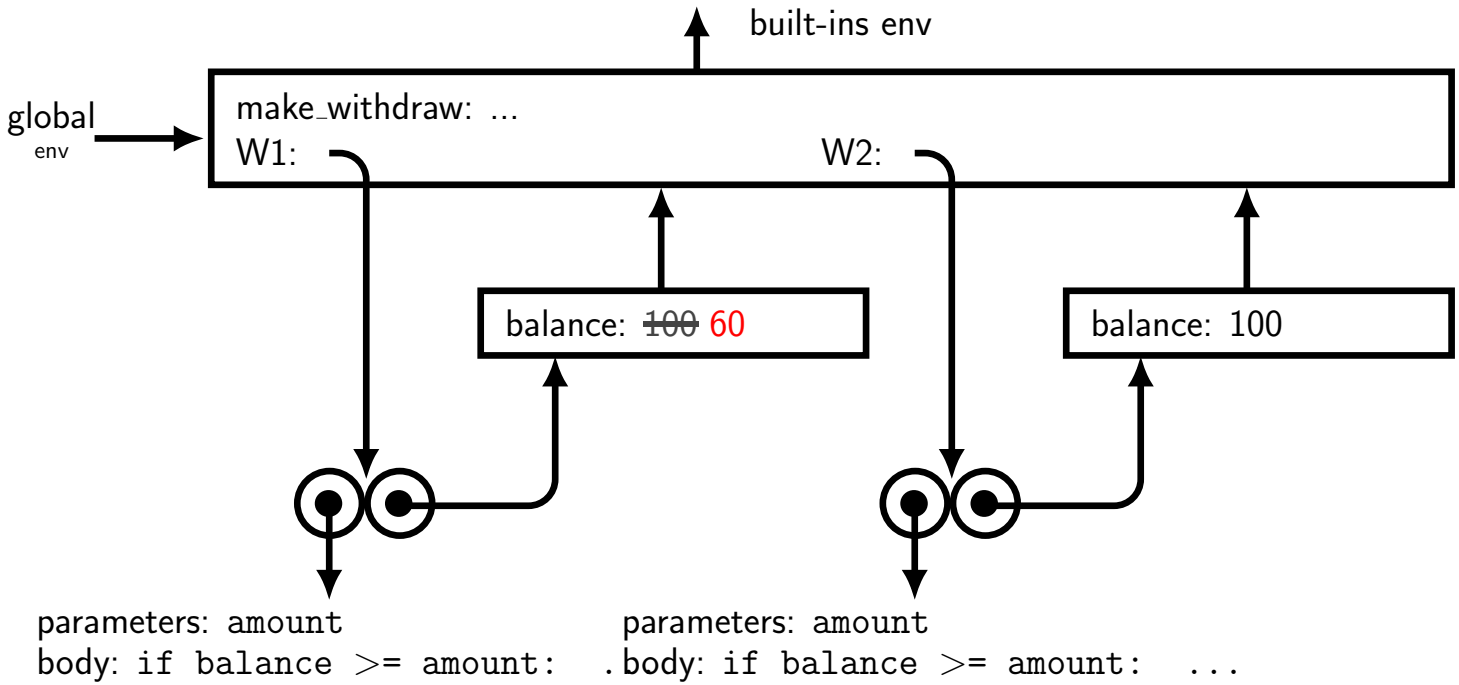
factorial Example

- Environment after evaluating `factorial(2)`.



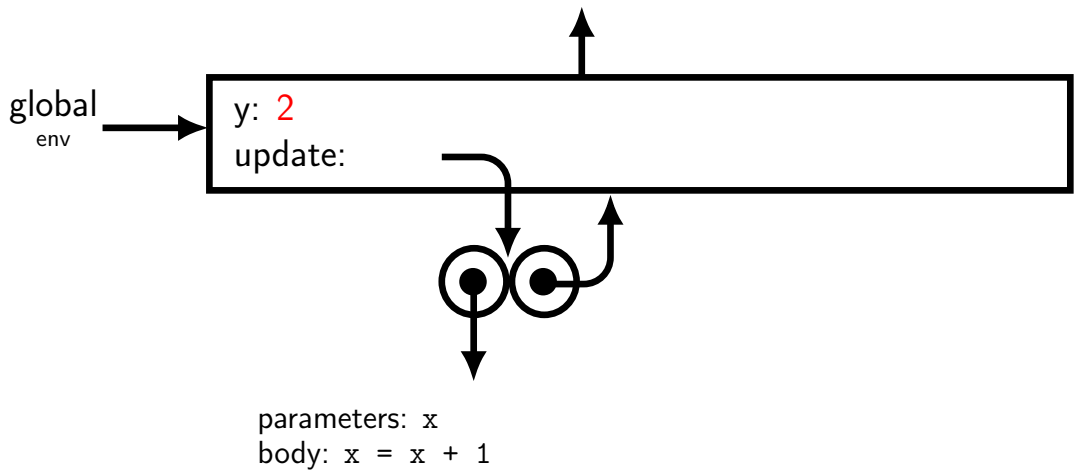
make_withdraw Example

- Environment after evaluating `W2 = make_withdraw(100)`.
- `W2` has its own state variable.



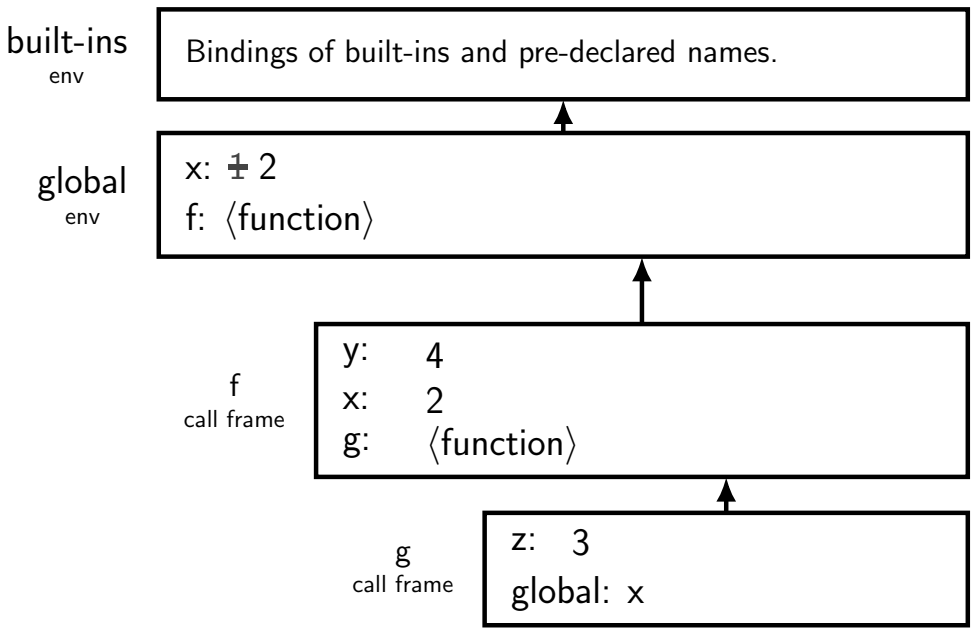
update Example

- Evaluate the argument `y` of `print(y)`.



The `global` Declaration

- `print(x)` prints 2: the global `x` changed, the `x` of `f` did not.




```
x = 1
def f(y):
    x = 2
    def g(z):
        global x
        x = x + 1
    g(3)
print(x)
f(4)
print(x)
```

output: 1 2

pair_add_one Example

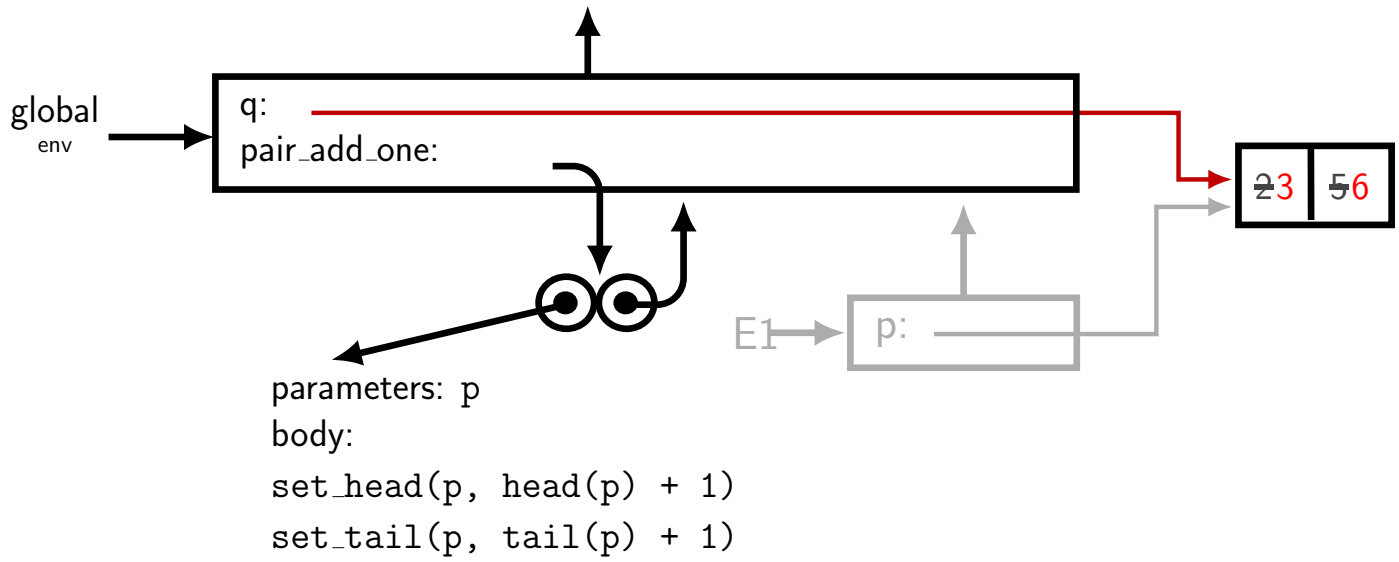
- What is the result of this program?

```
def pair_add_one(p):
    set_head(p, head(p) + 1)
    set_tail(p, tail(p) + 1)
q = pair(2, 5)
pair_add_one(q)
print(q)
```



pair_add_one Example

- Evaluate the argument `q` of `print(q)`.



Sharing and Identity

- `is` checks whether two names refer to the same pair.

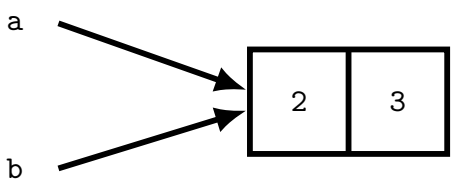
```
a = pair(2, 3)
b = a
print(a is b)
```



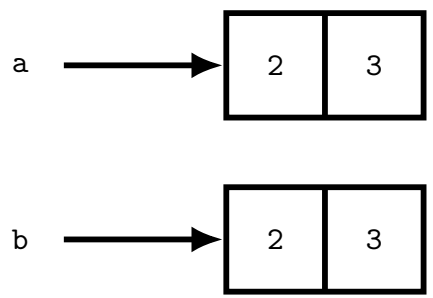
```
a = pair(2, 3)
b = pair(2, 3)
print(a is b)
```



`True`



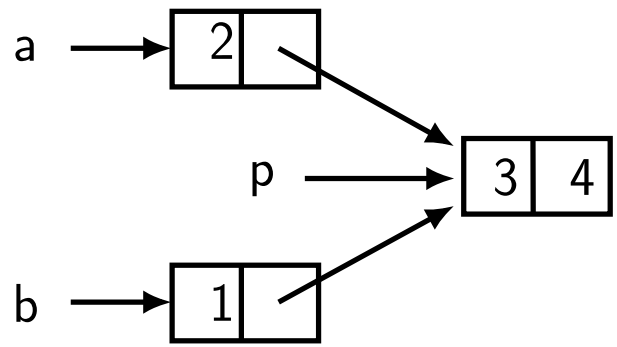
`False`



Sharing and Identity

- Larger structures can share a pair.

```
p = pair(3, 4)
a = pair(2, p)
b = pair(1, p)
print(tail(a) is tail(b))
```

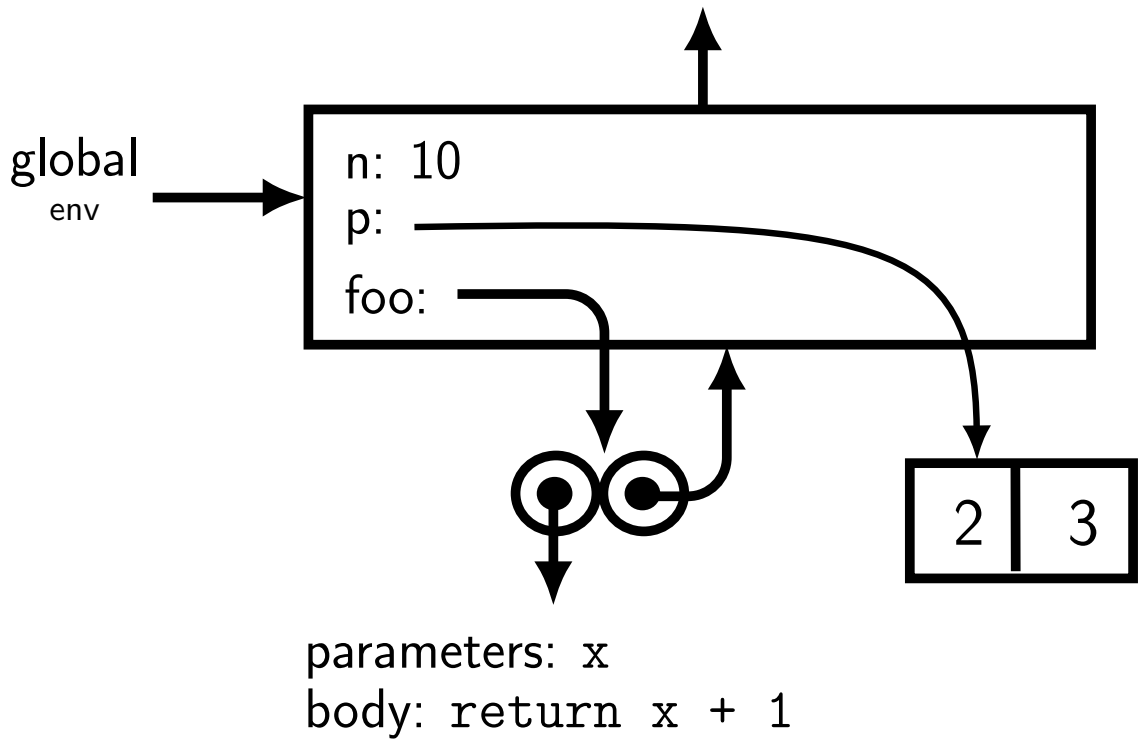


Identity in Python §3: is

- `a is b` asks whether `a` and `b` are the same object.
  - `True`, `False`, and `None` are each identical only to themselves.
  - Use == to compare numbers; `is` tests object identity.
  - Use == to compare strings by their characters.
  - Each function created by a lambda expression has a unique identity.
  - Each pair created by `pair` has a unique identity.
  - Each list created by a list expression has a unique identity.

Drawing Compound Structures

- Primitive values in bindings are drawn inside frames.
- Bindings point to compound objects drawn outside the frames, making identity and sharing visible.



Summary of Environment Model

- An environment is a sequence of frames containing name bindings.
- A function object contains its parameters, body, and defining environment.
- Applying a function extends its defining environment with a call frame.
- Call frame binds parameters to arguments; local variables are initially unassigned.
- Name lookup searches the current frame, then enclosing environments.
- Bindings can point to compound objects; shared arrows represent shared identity.